



Java访问本地文件系统

北京理工大学计算机学院
金旭亮

文件与文件系统



文件用于存放大量的数据，当程序运行结束时其数据可以被永久地保存在文件中。



文件可以保存在各种存储器中，硬盘、光盘、U盘.....，操作系统负责从存储器中装载和保存文件。



文件组织为文件夹，称为“**目录 (Directory)**”，目录可以嵌套，从而构成一棵多叉树。操作系统负责维护它们。



JVM提供了相应的API，Java应用程序可以通过它访问本地文件系统。

JDK中操作文件



与文件操作相关的类，集中于`java.io`包中。



Java使用`File`类来统一操作文件和文件夹。

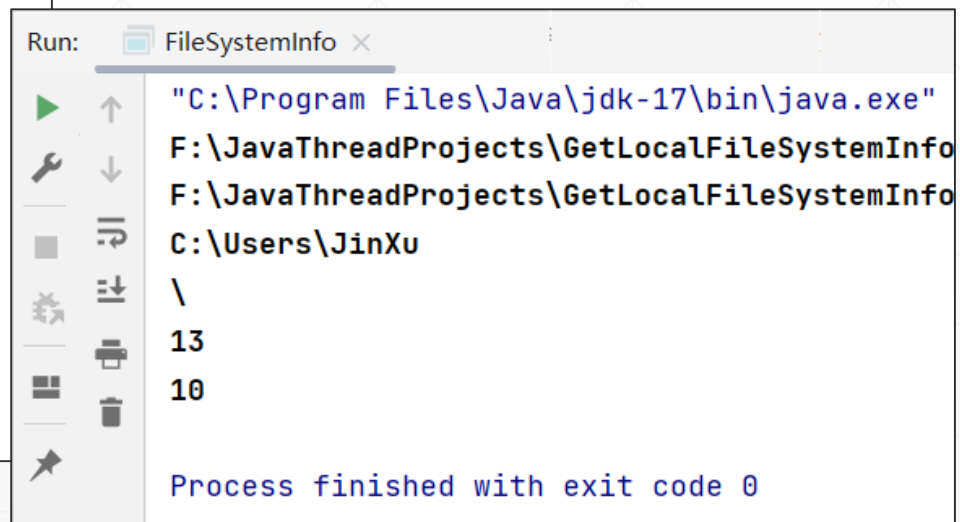


`File`类本质上是对文件或文件夹相关信息的封装，它并不真正打开或存取文件。

获取文件系统的一些有用信息

```
private static void getFileSystemInfo() throws IOException {  
    //获取用户当前工作目录  
    System.out.println(System.getProperty("user.dir"));  
    //另一种方式:  
    System.out.println(new File(" ").getCanonicalPath());  
    //获取用户目录  
    System.out.println(System.getProperty("user.home"));  
  
    //获取当前操作系统的路径分隔符  
    System.out.println(File.separator);  
  
    //获取当前操作系统的换行符  
    var separator : String = System.getProperty("line.separator");  
    //由于换行符不可见, 所以, 这里输出它的ASCII码值  
    separator.chars().forEach(System.out::println);  
}
```

FileSystemInfo.java



The screenshot shows a Java IDE's Run console window titled "Run: FileSystemInfo x". The output of the program is as follows:

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
F:\JavaThreadProjects\GetLocalFileSystemInfo  
F:\JavaThreadProjects\GetLocalFileSystemInfo  
C:\Users\JinXu  
\br/>13  
10  
  
Process finished with exit code 0
```

获取操作系统相关的信息



//获取当前操作系统的相关参数信息

```
private static void getAllSystemProperties() {  
    System.getProperties().forEach((key, value) -> {  
        System.out.println(key + ":" + value);  
    });  
}
```

FileSystemInfo.java



Run: FileSystemInfo x

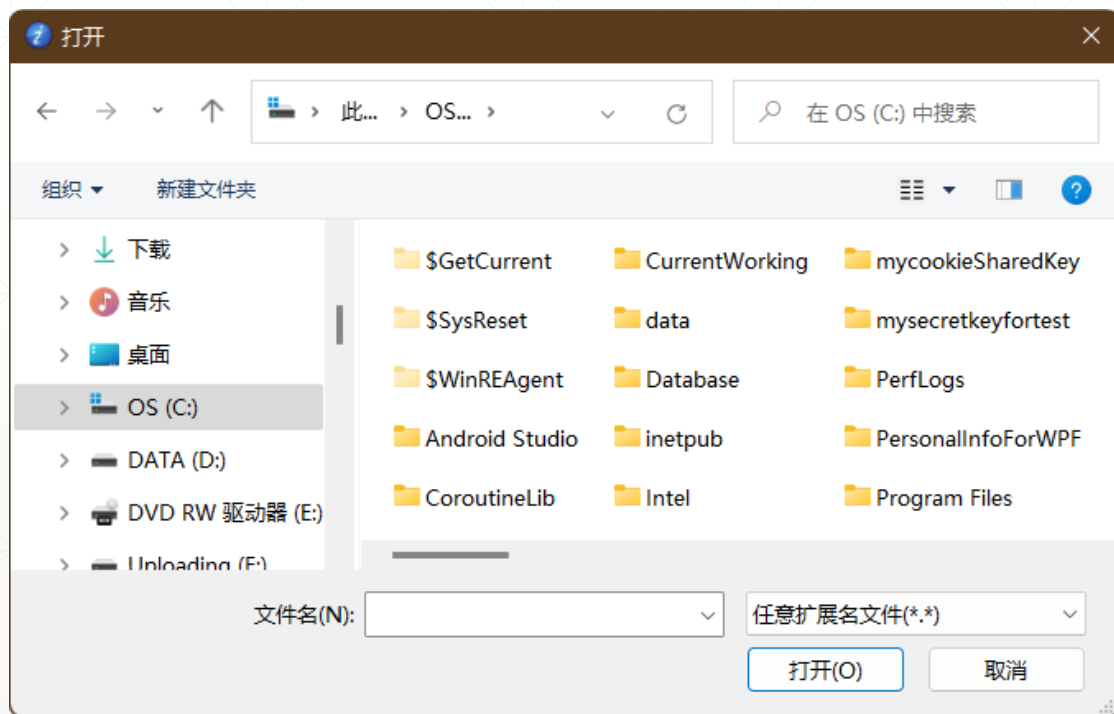
```
java.vm.specification.version:17  
os.name:Windows 10  
sun.java.launcher:SUN_STANDARD  
user.country:CN  
sun.boot.library.path:C:\Program Files\Java\jdk-17\bin  
sun.java.command:FileSystemInfo  
jdk.debug:release  
sun.cpu.endian:little  
user.home:C:\Users\JinXu  
user.language:zh  
java.specification.vendor:Oracle Corporation  
java.version.date:2021-09-14  
java.home:C:\Program Files\Java\jdk-17
```



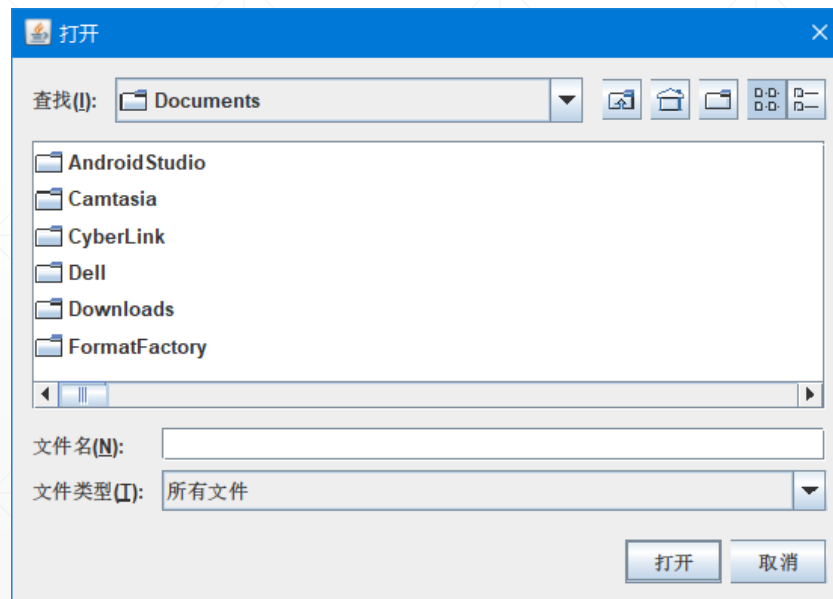
上述方法会在控制台窗口输入很多的信息项，建议同学们浏览一下，看看可以获取到哪些信息.....

JDK中的系统文件组件

操作系统通常都提供一些标准的文件对话框，比如打开和保存文件，但在Java中，并不直接使用这些特定于操作系统的对话框组件，而是提供了自己的对应组件，以便实现跨平台。



Windows 打开文件对话框



Java 打开文件对话框

Java中如何选择文件?

使用JFileChooser类，这个类位于Swing包中：

```
public class ChooseFile {  
    public static void main(String[] args) {  
        JFileChooser fileChooser = new JFileChooser();  
        fileChooser.setFileSelectionMode(  
            JFileChooser.FILES_ONLY);  
        //显示打开文件对话框  
        int result = fileChooser.showOpenDialog( parent: null);  
        // 如果用户点击了取消按钮.....  
        if (result == JFileChooser.CANCEL_OPTION)  
            return;  
        File fileName = fileChooser.getSelectedFile();  
        System.out.println("您选择了: " + fileName.getName());  
    }  
}
```

实例: ChooseFile.java

这个对话框是Swing组件，由于JDK中始终包容有Swing，所以它总是可用的，这点不像JavaFX，需要引用单独的外部模块才能使用。

获取磁盘分区相关信息

```
public class PartitionSpace {  
    public static void main(String[] args) {  
        File[] roots = File.listRoots();  
        for (File root : roots) {  
            System.out.println("Partition: " + root);  
            System.out.println("自由空间 (Free Space) = " +  
                root.getFreeSpace());  
            //getUsableSpace()方法会考虑到操作系统的特性, 给出更精确的结果  
            //详情请查看本方法的注释  
            System.out.println("可用空间 (Usable Space) = " +  
                root.getUsableSpace());  
            System.out.println("总容量 = " +  
                root.getTotalSpace());  
            System.out.println("***");  
        }  
    }  
}
```



Run: PartitionSpace x

"C:\Program Files\Java\jdk-17\bin\java.exe

Partition: C:\

自由空间 (Free Space) = 32442740736

可用空间 (Usable Space) = 32442740736

总容量 = 493758705664

Partition: D:\

自由空间 (Free Space) = 261899112448

可用空间 (Usable Space) = 261899112448

总容量 = 1000068870144

Partition: F:\

自由空间 (Free Space) = 225039069184

可用空间 (Usable Space) = 225039069184

总容量 = 480098873344